

Learning Equilibrium Polymer Distributions with Continuous Normalizing Flows and Diffusion Models

Author Name

Affiliation

(Dated: May 10, 2026)

Our long-term goal is to learn models that can support molecular trajectory inference from static structural samples, including protein conformational ensembles. We first study controlled Brownian-dynamics synthetic systems where the equilibrium distribution and score field can be evaluated directly. The codebase uses Rouse-chain simulations as the main data source and compares two likelihood- or score-based models: an equivariant continuous normalizing flow (CNF) and an equivariant variance-preserving diffusion model. Both models are trained on centered coordinate snapshots and are evaluated by generated-configuration quality and agreement between learned log-density gradients and Brownian drift forces.

I. MOTIVATION

Our long-term goal is to develop generative models that can help infer molecular conformational dynamics from static structural samples. This problem is especially relevant for proteins, where experimental methods can provide structural ensembles but often do not provide the time ordering or transition information needed to reconstruct dynamics directly. Directly evaluating this idea on atomistic proteins is difficult because the effective dynamics are not fully observed and the correct score or force field is not available as a clean target.

We therefore begin with controlled Brownian-dynamics synthetic systems. In this setting, equilibrium samples can be generated from a known stochastic process, and the corresponding score field is available for evaluation. This lets us test whether continuous normalizing flows and diffusion models can learn both the equilibrium distribution of configurations and the local log-density gradients that govern overdamped dynamics.

For a configuration $x_t \in \mathbb{R}^{d \times N}$ with N beads in d spatial dimensions, the simulated dynamics are

$$dX_t = F(X_t) dt + \sqrt{2D} dW_t, \quad (1)$$

where D is the diffusion coefficient, W_t is standard Brownian motion, and $F(x)$ is the deterministic drift. For the current base Rouse simulations, $D = 0.25$. For the Rouse-chain experiments, the base drift is generated by adjacent harmonic springs,

$$F_i(x) = k_\xi (x_{i+1} - 2x_i + x_{i-1}), \quad (2)$$

with endpoint modifications. The implementation also supports nonideal terms, including Lennard-Jones interactions, excluded volume, confinement, hairpin contacts, and ring closure.

At equilibrium, the drift is related to the score of the stationary density:

$$F(x) = D \nabla_x \log p_{\text{eq}}(x). \quad (3)$$

This relation gives a practical test for learned generative models. If a model learns $p_{\text{eq}}(x)$, then its learned

score should align with the Brownian drift divided by D . For the current base Rouse setting, the score target is therefore $F(x)/0.25 = 4F(x)$. The current experiments therefore focus on learning equilibrium coordinate distributions from static snapshots, then comparing generated samples and learned score fields against known Rouse-chain quantities.

II. HAMILTONIAN AND BOLTZMANN BACKGROUND

For atomistic molecular systems, the longer-term trajectory-inference framing is usually written in Hamiltonian form. Let $\vec{q}$ denote particle positions and $\vec{p}$ denote momenta. Hamilton's equations are

$$\frac{dq_{i,k}}{dt} = \frac{\partial H}{\partial p_{i,k}}, \quad (4)$$

$$\frac{dp_{i,k}}{dt} = -\frac{\partial H}{\partial q_{i,k}}, \quad (5)$$

where i indexes particles and k indexes spatial coordinates. For standard molecular dynamics, momenta satisfy

$$p_{i,k} = m_i \frac{dq_{i,k}}{dt}, \quad (6)$$

and the Hamiltonian is decomposed into kinetic and potential energy:

$$H(\vec{q}, \vec{p}, t) = KE(\vec{p}) + PE(\vec{q}) = \sum_i \sum_k \frac{p_{i,k}^2}{2m_i} + V(\vec{q}). \quad (7)$$

At thermal equilibrium, the probability of a configuration is given by the Boltzmann distribution

$$P(\vec{q}) = \frac{1}{Z} \exp\left(-\frac{V(\vec{q})}{k_B T}\right), \quad (8)$$

with partition function

$$Z = \int \exp\left(-\frac{V(\vec{q})}{k_B T}\right) d\vec{q}. \quad (9)$$

Taking a gradient with respect to $q_{i,k}$ gives

$$\begin{aligned} \frac{dp_{i,k}}{dt} &= -\frac{\partial H}{\partial q_{i,k}} = -\frac{\partial}{\partial q_{i,k}} V(\vec{q}) \\ &= k_B T \frac{\partial}{\partial q_{i,k}} \log P(\vec{q}). \end{aligned} \quad (10)$$

Thus, if a model can evaluate or differentiate the equilibrium log density, it can provide a force-like quantity. This is the bridge from generative density modeling to trajectory inference. In the present codebase, this idea is tested first in the overdamped Brownian setting, where the corresponding relation is $F(x) = D \nabla_x \log p_{\text{eq}}(x)$, or equivalently $\nabla_x \log p_{\text{eq}}(x) = F(x)/D$.

III. DATA

Training data are centered bead configurations sampled from simulated Rouse trajectories. The main data path loads HDF5 files containing arrays of shape

$$x \in \mathbb{R}^{d \times N \times B}, \quad (11)$$

where B is the number of sampled frames. The current polymer configs use two-dimensional chains with $N = 32$ beads. For the base Rouse simulation, trajectories are generated by the SDE above with independent Brownian noise in every bead coordinate. The base simulation uses $D = 0.25$, unit bond length, and $k_\xi = 3.0$. The saved raw trajectories may include center-of-mass diffusion, but model training centers each frame so the models learn internal conformations rather than global translation.

IV. TIME CONVENTIONS

The manuscript uses three time variables with different meanings. In the Brownian dynamics simulator, t is physical simulation time. It indexes the stochastic trajectory used to generate equilibrium snapshots. In the CNF, we use $\tau \in [0, 1]$ as a model pseudotime: $\tau = 0$ is the data distribution and $\tau = 1$ is the standard normal base distribution. Likelihood evaluation integrates from $\tau = 0$ to $\tau = 1$, while sampling integrates back from $\tau = 1$ to $\tau = 0$. In the diffusion model, $t \in [0, 1]$ is noising time: $t = 0$ is clean data and $t = 1$ is approximately standard Gaussian noise. The reverse diffusion sampler starts at $t = 1$ and integrates to a small positive time, currently 10^{-4} , for numerical stability. These conventions match the implementation.

V. SYNTHETIC SYSTEMS

We focus on two controlled synthetic systems: the base Rouse chain and a hairpin-stabilized Rouse chain.

The base system is the simplest test of whether a model can learn the equilibrium distribution and score of a flexible Brownian polymer. The hairpin system adds local structure through attractive nonlocal contacts and repulsive excluded volume. It is not intended to be a protein model, but it gives a minimal analogue of a stable folded motif: a chain has flexible thermal motion, but a subset of long-range contacts stabilizes a preferred conformation.

Base Rouse Chain

The base Rouse model uses a quadratic bond potential between adjacent beads. In the notation of the score model, the dimensionless equilibrium potential is

$$U_{\text{bond}}(x) = \frac{k_\xi}{2D} \sum_{i=2}^N \|x_i - x_{i-1}\|^2. \quad (12)$$

The corresponding equilibrium score is

$$\nabla_x \log p_{\text{eq}}(x) = -\nabla_x U_{\text{bond}}(x), \quad (13)$$

and the Brownian drift is $F(x) = D \nabla_x \log p_{\text{eq}}(x)$. The base configuration uses $N = 32$ beads in two dimensions, $D = 0.25$, and $k_\xi = 3.0$. All nonideal interactions are disabled in this setting. This makes the base Rouse chain a clean baseline because the equilibrium score is generated entirely by adjacent harmonic springs.

Nonideal Interactions

The simulator also supports several nonideal potentials. For nonbonded Lennard-Jones interactions, the potential has the form

$$U_{\text{LJ}}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right], \quad (14)$$

with optional cutoff, softening, bonded-neighbor exclusion, and energy shifting at the cutoff. The force implementation uses the corresponding radial derivative with softened squared distance $r^2 + \delta^2$ for numerical stability.

Excluded volume uses a purely repulsive power-law potential,

$$U_{\text{EV}}(r) = \epsilon_{\text{EV}} \left(\frac{\sigma_{\text{EV}}}{r} \right)^p, \quad (15)$$

again with cutoff, softening, and bonded-neighbor exclusion. In the current excluded-volume configuration, the defaults inherited from the base file are

$$\begin{aligned} \epsilon_{\text{EV}} &= 0.03, & \sigma_{\text{EV}} &= 1.0, & p &= 12, \\ \delta &= 0.3, & r_{\text{cut}} &= 1.5. \end{aligned} \quad (16)$$

The purpose is to prevent physically implausible bead overlap while preserving the stochastic flexibility of the chain.

Hairpin-Stabilized Chain

The hairpin system starts from a two-strand initialization. For an even number of beads, the first half of the chain is initialized on one strand and the second half on the opposing strand, with paired beads facing each other. The current hairpin configuration uses an initial strand separation of 0.95 and Gaussian initialization noise with standard deviation 0.02.

Hairpin stabilization is implemented by attractive Lennard-Jones contacts between paired beads:

$$i + j = N + 1, \quad |i - j| \geq s_{\min}. \quad (17)$$

Only these designated long-range pairs receive the hairpin LJ interaction. The current parameters are

$$\epsilon_{\text{hp}} = 5.0, \quad \sigma_{\text{hp}} = 0.9, \quad \delta_{\text{hp}} = 0.3, \\ r_{\text{cut}} = 3.0, \quad s_{\min} = 2. \quad (18)$$

The hairpin configuration also enables excluded volume and centered confinement. The confinement potential is

$$U_{\text{conf}}(x) = c \sum_{i=1}^N \|x_i - \bar{x}\|^2, \quad c = 0.02, \quad (19)$$

where $\bar{x}$ is the chain center of mass. Together, these terms create a stable but thermally fluctuating structure. This is the synthetic example closest to the protein motivation: the model must learn not only a broad polymer ensemble, but also a distribution shaped by stabilizing nonlocal contacts.

VI. CONTINUOUS NORMALIZING FLOW

The CNF model learns an ODE that maps data coordinates to a standard normal base distribution. Let $z(\tau)$ denote the CNF state at pseudotime $\tau \in [0, 1]$:

$$\frac{dz(\tau)}{d\tau} = f_{\theta}(z(\tau), \tau). \quad (20)$$

Here, τ is a generative-model coordinate, not physical Brownian time. The implemented CNF integrates from data to noise for likelihood evaluation and from noise to data for sampling.

The log-density evolves according to the instantaneous change-of-variables formula:

$$\frac{d}{d\tau} \log p(z(\tau)) = -\text{Tr} \left(\frac{\partial f_{\theta}}{\partial z} \right). \quad (21)$$

Equivalently, the accumulated log-density change is

$$\log P(z(\tau)) - \log P(z(0)) = \Delta \log P_z(\tau) \\ = \int_0^{\tau} -\text{Tr} \left(\frac{\partial f_{\theta}}{\partial z(\lambda)} \right) d\lambda. \quad (22)$$

For a training sample x , the model solves

$$z(0) = x, \quad (23)$$

$$z(1) = \text{ODESolve}(z(0), f_{\theta}, 0, 1). \quad (24)$$

The log likelihood is then

$$\log p_{\theta}(x) = \log \mathcal{N}(z(1); 0, I) - \Delta \log p, \quad (25)$$

where $\Delta \log p$ is accumulated during the ODE solve. The same calculation can be written as the augmented ODE system

$$\begin{bmatrix} z(1) \\ \Delta \log P_z(1) \end{bmatrix} = \begin{bmatrix} z(0) = x \\ \Delta \log P_z(0) = 0 \end{bmatrix} \\ + \int_0^1 \begin{bmatrix} f_{\theta}(z(\tau), \tau) \\ -\text{Tr} \left(\frac{\partial f_{\theta}}{\partial z(\tau)} \right) \end{bmatrix} d\tau. \quad (26)$$

The training objective is negative log likelihood:

$$\mathcal{L}_{\text{CNF}}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log p_{\theta}(x)]. \quad (27)$$

The current polymer CNF uses an EGNN-style vector field. The network builds pairwise messages from node embeddings, sequence separation $i - j$, squared pair distances $\|x_i - x_j\|^2$, and pseudotime τ . Coordinate updates are formed from relative displacements, so the vector field is translation equivariant after centering. For the main EGNN CNF configuration, the divergence term is estimated with a Hutchinson JVP trace estimator rather than an exact trace. Sampling starts from Gaussian noise and integrates the same CNF dynamics backward from $\tau = 1$ to $\tau = 0$.

Adjoint Sensitivity Equations

The original CNF formulation can compute parameter gradients with an adjoint ODE. For the scalar objective

$$\mathcal{L} = \log P(z(1)) - \Delta \log P_z(1), \quad (28)$$

define the adjoint state

$$a(\tau) = \begin{bmatrix} a_z(\tau) \\ a_{\Delta}(\tau) \end{bmatrix} = \begin{bmatrix} \partial \mathcal{L} / \partial z \\ \partial \mathcal{L} / \partial (\Delta \log P_z) \end{bmatrix}. \quad (29)$$

At $\tau = 1$,

$$a(1) = \begin{bmatrix} \nabla_z \log P(z(1)) \\ -1 \end{bmatrix}. \quad (30)$$

The adjoint dynamics are

$$\frac{da(\tau)}{d\tau} = - \begin{bmatrix} \frac{\partial f_{\theta}}{\partial z} & 0 \\ -\nabla_z \text{Tr} \left(\frac{\partial f_{\theta}}{\partial z} \right) & 0 \end{bmatrix}^T a(\tau), \quad (31)$$

$$\frac{d}{d\tau} \left(\frac{\partial \mathcal{L}}{\partial \theta} \right) = a(\tau)^T \begin{bmatrix} \frac{\partial f_{\theta}}{\partial \theta} \\ -\nabla_{\theta} \text{Tr} \left(\frac{\partial f_{\theta}}{\partial z} \right) \end{bmatrix}. \quad (32)$$

The Julia implementation uses differentiable ODE solves and adjoint machinery through the SciML stack rather than manually coding these equations in the experiment loop.

VII. VARIANCE-PRESERVING DIFFUSION MODEL

The diffusion baseline learns the score field directly. The implementation uses a continuous-time variance-preserving (VP) forward SDE

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)} dw_t, \quad (33)$$

with a linear noise-rate schedule

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min}), \quad t \in [0, 1], \quad (34)$$

with $\beta_{\min} = 0.1$ and $\beta_{\max} = 20$ in the code. The marginal forward process is

$$x_t = \mu(t)x_0 + \sigma(t)\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (35)$$

where

$$\mu(t) = \exp \left[-\frac{1}{4}(\beta_{\max} - \beta_{\min})t^2 - \frac{1}{2}\beta_{\min}t \right], \quad (36)$$

$$\sigma^2(t) = 1 - \exp \left[-\frac{1}{2}(\beta_{\max} - \beta_{\min})t^2 - \beta_{\min}t \right]. \quad (37)$$

Both x_0 and the sampled noise are centered, so the diffusion process stays in the centered coordinate subspace used by training. For numerical stability, training times are clamped below by 10^{-5} and $\sigma(t)$ is floored by the corresponding minimum positive value before forming the score target.

The score network $s_\theta(x_t, t)$ is trained to estimate

$$\nabla_{x_t} \log p_{0t}(x_t|x_0) = -\frac{\epsilon}{\sigma(t)}. \quad (38)$$

The implemented loss is the weighted denoising score-matching objective

$$\mathcal{L}_{\text{diff}}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[\sigma^2(t) \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma(t)} \right\|_2^2 \right]. \quad (39)$$

The diffusion score model also uses an EGNN-style architecture. It keeps learned node embeddings, uses sequence-distance and radial pair features, and updates coordinates through pairwise relative displacements. The output is centered and interpreted directly as $\nabla_x \log p_t(x)$.

Generation uses the reverse-time VP SDE. For the implemented linear schedule, the reverse coefficients are

$$f(t) = -\frac{1}{2}\beta(t), \quad g(t)^2 = \beta(t). \quad (40)$$

Starting from centered Gaussian noise at $t = 1$, the Euler-Maruyama sampler steps backward to a small positive time:

$$x_{t-\Delta t} = x_t - [f(t)x_t - g(t)^2 s_\theta(x_t, t)] \Delta t + g(t)\sqrt{\Delta t}z, \quad z \sim \mathcal{N}(0, I). \quad (41)$$

The sampler integrates from $t = 1$ to $t = 10^{-4}$ by default. For numerical stability, it clips sample score norms to 64, recenters each state, checks for non-finite samples, and omits the final noise draw on the last step. During diffusion training, parameter gradients are also clipped elementwise to magnitude 1.

VIII. EVALUATION

The experiments report sample-level statistics and score-field diagnostics. For generated samples, the main structural metric is the mean absolute error between average pairwise distances in generated configurations and training configurations. The code also records the fraction of finite generated values and finite generated samples.

For Rouse and polymer Langevin data, the simulator provides analytic force targets. The CNF score is obtained by differentiating the learned log likelihood,

$$\nabla_x \log p_\theta(x), \quad (42)$$

while the diffusion score is the network output at a small time value near zero. These scores are compared with the known Brownian score target $F(x)/D$, with normalization corrections applied when the data normalizer is enabled. This makes the benchmark more direct than visual sample comparison alone: the model should not only generate plausible chain conformations, but also recover the local equilibrium score implied by the Brownian dynamics.

IX. JULIA AND SCIML IMPLEMENTATION

The implementation is built around the SciML ecosystem. This is not only a convenience choice: the project needs stochastic simulation, neural ODE likelihoods, differentiable ODE solves, GPU-compatible array code, and several trace-estimation strategies in the same code path. Julia makes these pieces composable without translating between separate simulation, autodiff, and machine-learning frameworks.

Brownian Data Generation with StochasticDiffEq

The Rouse data generator is written directly as an SDE problem. The simulator constructs

$$dX_t = F(X_t) dt + \sqrt{2D} dW_t \quad (43)$$

as a `SDEProblem`, with the drift function calling the same polymer force routine used later for score evaluation. The raw trajectory script uses `StochasticDiffEq` solvers through a configurable solver interface. The available choices include Euler-Maruyama, Euler-Heun, RKMil, SOSRA, and SOSRA2. This lets the same

model family generate inexpensive baseline trajectories or more stable stochastic trajectories by changing a YAML configuration rather than rewriting simulation code.

The saved trajectories are then reused by the learning experiments through HDF5 loaders. This gives a clean separation between stochastic simulation and generative-model training while keeping both stages inside Julia.

CNF Likelihoods as SciML ODE Problems

The CNF is implemented as an `ODEProblem` whose state is a `ComponentArray` containing both coordinates and accumulated log-density change:

$$u(\tau) = (x(\tau), \Delta \log p(\tau)). \quad (44)$$

The ODE right-hand side returns

$$\frac{d}{d\tau} \begin{bmatrix} x(\tau) \\ \Delta \log p(\tau) \end{bmatrix} = \begin{bmatrix} f_\theta(x(\tau), \tau) \\ -\text{Tr}(\partial f_\theta / \partial x) \end{bmatrix}. \quad (45)$$

The same problem definition is solved forward for likelihood evaluation and backward for sample generation. The solver is selected from the SciML ODE stack, with current configs using adaptive `Tsit5` and support for fixed-step `RK4`. For training, the solve uses `InterpolatingAdjoint(autojacvec=ZygoteVJP())`, so gradients propagate through the ODE solve while remaining compatible with the neural vector field and the trace estimator.

This design keeps the generative model close to its mathematical definition. The code does not need a separate CNF framework: the likelihood model is an ODE with a structured state, a vector field, a trace estimator, and a SciML adjoint.

Hutchinson JVP Trace Estimator

The main EGNN CNF configuration uses a Hutchinson trace estimator with a Jacobian-vector product (JVP). For a vector field $f_\theta(x, \tau)$, the divergence can be estimated as

$$\text{Tr} \left(\frac{\partial f_\theta}{\partial x} \right) = \mathbb{E}_v \left[v^T \frac{\partial f_\theta}{\partial x} v \right], \quad (46)$$

where v is a Rademacher probe. The implementation samples one probe per batch and computes

$$\hat{\text{Tr}} = \sum_{i,j} v_{i,j} [J_{f_\theta}(x, \tau) v]_{i,j}. \quad (47)$$

The important engineering step is that the EGNN JVP is propagated explicitly through the vector field. The

implementation carries both the primal coordinate update and the directional derivative through pairwise displacements, squared distances, edge networks, node updates, and coordinate aggregation. For example, if $r_{ij} = x_i - x_j$, then the directional derivative carried by the JVP is

$$dr_{ij} = v_i - v_j, \quad d\|r_{ij}\|^2 = 2r_{ij}^T dr_{ij}. \quad (48)$$

These directional features are passed through the same edge and node networks as the primal features, producing both $f_\theta(x, \tau)$ and $J_{f_\theta}(x, \tau)v$ in one structured forward pass. This avoids materializing the full Jacobian over all bead coordinates. For a 32-bead chain in two dimensions, the full Jacobian is already a dense object over 64 coordinate variables per sample; for larger systems, exact traces become the wrong abstraction. The JVP estimator keeps the CNF likelihood scalable while preserving the same ODE interface.

The code also keeps alternative trace modes. An exact Zygote-based divergence is available for smaller checks. There is also a finite-difference Hutchinson mode, `hutchinson_fd`, which approximates the same JVP by

$$J_{f_\theta}(x, \tau)v \approx \frac{f_\theta(x + \epsilon v, \tau) - f_\theta(x - \epsilon v, \tau)}{2\epsilon}. \quad (49)$$

This finite-difference path is useful as a diagnostic fallback because it tests the same trace estimator without relying on the analytic JVP implementation. The production path uses `hutchinson_jvp` because it is faster and analytic: it avoids two extra vector-field evaluations per trace estimate, avoids finite-difference step-size error, and remains differentiable through the exact directional-derivative computation. Zygote is still used for training gradients through the CNF loss and SciML adjoint solve, but it is not deriving this EGNN JVP automatically; the JVP is implemented explicitly in the model code.

Autodiff and Stability

Zygote is used for reverse-mode training gradients through both CNF and diffusion losses. Several small custom adjoints remove avoidable AD overhead for common tensor operations such as batched node embeddings, row concatenation, and centering. The diffusion implementation also uses `Zygote.ignore` and `dropgrad` where fixed geometric inputs, masks, sampled noise, or numerical guards should not become part of the trainable derivative path.

The result is an implementation where stochastic simulation, adaptive ODE solving, adjoint differentiation, neural network training, and GPU execution remain in one language and one type system. That is the main Julia advantage for this project: the mathematical objects in the paper map directly to executable SciML problems, and the engineering choices needed for stability remain

visible in the code rather than hidden behind a separate modeling layer.

X. TRAJECTORY-INFERENCE ALGORITHM

The full long-term BoltzFlow trajectory-inference idea from the earlier proposal can be summarized as follows. First, obtain equilibrium snapshots

$$\{\vec{q}_1, \dots, \vec{q}_n\} \sim P(\vec{q}). \quad (50)$$

Then train a CNF to model $P(\vec{q})$ and evaluate $\log P(\vec{q})$. For a trajectory initialized at $\vec{q}(t_0)$, sample momenta from the Maxwell-Boltzmann distribution,

$$p_{i,k}(t_0) \sim \mathcal{N}(0, m_i k_B T). \quad (51)$$

At each physical timestep, compute the CNF log density and its gradient:

$$\nabla_{\vec{q}} \log P(\vec{q}(t)). \quad (52)$$

Using the Boltzmann relation,

$$-\nabla_{\vec{q}} V(\vec{q}) = k_B T \nabla_{\vec{q}} \log P(\vec{q}), \quad (53)$$

update the momenta and positions with Hamilton's equations:

$$\frac{d\vec{p}}{dt} = k_B T \nabla_{\vec{q}} \log P(\vec{q}), \quad (54)$$

$$\frac{d\vec{q}}{dt} = \frac{\partial H}{\partial \vec{p}}. \quad (55)$$

This Hamiltonian trajectory-inference loop is the protein-facing motivation. The current implementation does not yet execute this loop for atomistic proteins; it instead validates the score-learning components on Brownian/Rouse systems where the target force field is known.

XI. CURRENT SCOPE

The current codebase should be described as a Brownian-dynamics and polymer-equilibrium study. Protein dynamics can remain a longer-term motivation, but the implemented system does not yet infer atomistic protein trajectories from static structural ensembles. The practical claim supported by the code is narrower: equivariant CNF and diffusion models can be trained on centered equilibrium snapshots from Rouse-style Brownian simulations, sampled as generative models, and evaluated against known equilibrium score fields.